Artifact Rendering Architecture

A Structured Debate

Templates vs. Dynamic Widget Ranking vs. Hybrid Template Grammar

AWS Observability Claude Plugin | April 2026

PM (Product Manager)	**Eng** (Principal Engineer)	**UX** (UX Designer)
User value, adoption, time-to-market	System boundaries, extensibility, failure modes	Mental models, hierarchy, consistency

Table of Contents

1. Round 1: Opening Positions	3
2. Round 2: Cross-Examination	6
3. Round 3: Convergence	9
4. Conclusion: Orchestrator Synthesis	11
4.1 Decision	11
4.2 Architecture Specification	11
4.3 Implementation Plan	13
4.4 Open Questions	13
4.5 Risk Register	14

Round 1: Opening Positions

Sarah Chen (Product Manager)

Let me ground this in where we actually are. We have six artifact templates that we hand-built over the past quarter. They work. Users see them. The investigation-summary and service-health-card get rendered in probably 80% of plugin interactions. My concern is that we are about to spend a sprint or more re-architecting something that is already shipping value.

My position: start with templates, iterate toward hybrid only when we hit a concrete wall. Here is why. First, our adoption metrics tell me users care about speed and accuracy of the diagnosis, not about the layout. Nobody has filed a feature request saying 'I wish the trace waterfall could also show a metric sparkline next to it.' They file requests saying 'the root cause was wrong' or 'it took too long.' Second, templates are deterministic. If I am a customer SRE staring at a P1 at 3 AM, I want to see the same shape of artifact every time so I know where to look. Muscle memory matters. Third, the maintenance cost of templates is linear and predictable: one new template per new use case. That is a cost I can plan for. A dynamic system has a non-linear maintenance curve because you are debugging emergent layout behavior, not just data mapping.

That said, I am not blind to the limitations. We are already seeing template proliferation: six now, probably twelve by end of year. If each one requires a designer and an engineer to build, that does not scale. So I am open to hybrid, but only if someone shows me that the implementation cost is comparable to building two or three more templates, and that the LLM can reliably pick the right shell and fill it without introducing layout regressions. Show me a prototype, not a design doc.

My main question for the room: what is the user-visible difference between a well-chosen template and a hybrid shell with the right widgets? If the answer is 'it looks basically the same,' then the engineering cost of the dynamic system is pure overhead.

Marcus Rivera (Principal Engineer)

Sarah is right that the current templates work, but she is framing this as a feature question when it is actually a systems architecture question. Let me reframe. The real question is: who owns layout decisions, and what is the contract between the LLM reasoning layer and the rendering layer?

Right now, the answer is 'nobody and nothing.' The LLM picks a template name from a hardcoded list, dumps JSON data into it, and the frontend renders it. There is no schema validation, no fallback behavior, no versioning. If a template expects a field called 'traceSpans' and the LLM outputs 'spans,' the artifact silently breaks. I have seen this in production three times this month.

Pure templates fail for a systems reason: the combinatorial space of observability queries is too large for a finite template set. Consider: a user asks 'why is my API slow?' The answer might involve traces, metrics, logs, and CloudTrail events, or just traces and a metric. The current system picks investigation-summary and crams everything in. But the information hierarchy is completely different for a latency investigation versus a permissions failure versus a deployment regression. We cannot design a template for every combination.

Pure widget ranking, the other extreme, is equally problematic. Giving the LLM full control over spatial layout means every response is a layout design problem. LLMs are bad at spatial reasoning. You will get overlapping widgets, inconsistent sizing, and accessibility nightmares. I have benchmarked this: GPT-4

and Claude both produce broken CSS grid layouts about 30% of the time when given free-form placement.

The hybrid approach works because it separates concerns correctly. The LLM makes semantic decisions (what information matters, in what order), and the renderer makes spatial decisions (how to lay it out within a constrained shell). The contract is a JSON manifest that specifies: which shell, which widgets go in which named slots, what data each widget receives. The renderer is deterministic given that manifest. This means layout bugs are reproducible, testable, and fixable without touching the LLM layer.

The schema I am proposing looks roughly like this: the manifest has a shell identifier, a metadata block with title and severity, and a slots object where each key is a named slot in the shell and the value is an array of widget descriptors. Each widget descriptor has a type from the catalog, a data payload, and optional display hints like size preference. The renderer validates the manifest against the shell schema and rejects or falls back gracefully if it is malformed.

Priya Nair (UX Designer)

I want to start with what users actually see, because I think both Sarah and Marcus are abstracting away the visual experience too quickly.

I have watched twelve screen-share sessions with SREs using the plugin over the past month. Here is what I observed. Users do not read artifacts top-to-bottom. They scan. They look for the red thing, the spike in the chart, the highlighted span in the waterfall. Their eyes follow a predictable pattern: severity indicator, then primary visualization, then supporting detail. When an artifact matches that scan pattern, time-to-insight is under 15 seconds. When it does not, they spend 30 to 45 seconds reorienting.

This tells me two things. First, Sarah is right that consistency matters, but she is wrong about what consistency means. It is not 'the same template every time.' It is 'the information hierarchy is predictable.' Users do not memorize template shapes; they memorize where to look for specific types of information. A severity badge should always be top-left. The primary visualization should always be in the dominant content area. Supporting context should be secondary.

Second, Marcus is right that we need dynamic content, but the dynamism should be at the widget level, not the layout level. I would define four shells based on user intent, not data type. A single-focus shell for 'tell me about this one thing' queries. An investigation shell for 'why is this broken' queries. A comparison shell for 'what changed between X and Y.' And a dashboard shell for 'show me the current state of everything.' Each shell has a fixed visual hierarchy with named regions: a header zone, a primary content zone, a supporting context zone, and an optional detail drawer.

The widget catalog needs strict guardrails. No more than five widgets per view, because cognitive load research shows that beyond five distinct information chunks, comprehension drops sharply. Each widget should have a defined information density level from one to three: a stat card is density one, a sparkline is density two, a full trace waterfall is density three. The total density score per shell should not exceed a defined budget, and the shell should enforce this by collapsing lower-priority widgets into a 'show more' affordance. This is progressive disclosure applied to observability.

My biggest concern with the current templates is that they have no notion of priority or progressive disclosure. Everything is shown at full fidelity all the time. When an investigation has two relevant signals, that is fine. When it has eight, the artifact becomes a wall of charts that nobody can parse at 3 AM.

Round 2: Cross-Examination

Cross-Examination and Responses

Sarah Chen (PM challenges Engineer and UX)

Marcus, I have two pointed questions. First, you said the LLM outputs broken data-to-template mappings three times this month. How does adding a more complex manifest schema reduce that error rate rather than increase it? You are asking the LLM to output a richer JSON structure with shell IDs, slot names, widget types, data payloads, and display hints. That is strictly more surface area for the LLM to get wrong. What is your mitigation?

Second, you mentioned a validation layer that rejects malformed manifests. What happens when it rejects? The user asked a question and gets nothing? A fallback template? Because if the fallback is a template, you have just built an expensive system that degrades to what we have today, which means the real question is how often the dynamic path works correctly. Give me a number.

Priya, you mentioned four shells based on user intent. I like that framing, but intent classification is itself a hard LLM problem. How confident are you that we can reliably distinguish 'tell me about this one thing' from 'why is this broken'? In my experience, user queries are ambiguous: 'my API is slow' could be a single-focus on latency or an investigation into a cascading failure. If we get the shell wrong, the entire information hierarchy is wrong, which by your own argument is worse than a generic template that is merely adequate.

I also want to push back on the 'five widgets max' rule. That sounds like a design principle, not a product requirement. If a P1 investigation genuinely has eight relevant signals, hiding three of them behind a 'show more' button means the SRE has to click to find critical information. I would rather show all eight in a scrollable view than risk hiding a smoking gun.

Marcus Rivera (Engineer responds and challenges UX)

Sarah, fair challenges. Let me address them directly. On manifest complexity versus error rate: the key insight is that a structured schema is validatable, but a template name string is not. Today if the LLM outputs 'investigation_summary' instead of 'investigation-summary,' it fails silently. A JSON schema lets us validate structure, provide defaults for missing fields, and give the LLM structured error feedback for retry. In my testing with a draft schema, the LLM produces valid manifests 94% of the time on the first attempt, and 99% after one retry with the validation error message included in the prompt. That is better than our current template selection accuracy of about 88%.

On fallback behavior: yes, the fallback is a template, specifically the investigation-summary template, which is our most general layout. But the point is not that the fallback exists; it is that the fallback is an exceptional path, not the normal one. We log every fallback, which gives us a clear signal for which query patterns need better handling. With pure templates, there is no signal that a template was a bad fit. It just renders and the user silently has a subpar experience.

Now, Priya, I want to challenge your shell model. You proposed four shells. My concern is that the shell is load-bearing infrastructure: it defines the named slots, the CSS grid layout, the responsive breakpoints, and the widget sizing constraints. Each shell is probably 200 to 400 lines of tested frontend code. Four shells means 800 to 1600 lines of layout code that must be maintained in lockstep with the manifest

schema. How do we version this? If we add a new slot to the investigation shell, does every existing manifest that targets that shell break? I need you to define the contract between shell and widget more precisely.

Also, on information density budgets: I like the concept but it needs to be mechanically enforceable in the renderer, not advisory guidance for the LLM. Are you proposing that the renderer itself enforces density limits by truncating the widget list? Because if so, the LLM's ranking order becomes the priority signal, and the renderer just cuts at the budget. That is actually a clean separation of concerns.

Priya Nair (UX responds and challenges PM)

Let me address Marcus first, then come back to Sarah. On shell versioning: each shell should define a set of named slots with cardinality constraints, not a pixel-level layout. For example, the investigation shell defines: a header slot that accepts exactly one widget, a primary slot that accepts one to two widgets, a context slot that accepts zero to three widgets, and a detail slot that accepts zero to N widgets but renders collapsed by default. The CSS grid is an implementation detail owned by the frontend; the manifest contract is about slot names and cardinality. If we add a new slot, old manifests that do not reference it are unaffected. This is additive versioning, not breaking.

On density enforcement: yes, I am proposing that the renderer enforces it mechanically. The LLM ranks widgets by relevance and the renderer places them in priority order until the density budget is exhausted. Remaining widgets go into the collapsed detail slot. This means the LLM never needs to reason about visual layout at all. It just says 'here are the widgets I think matter, in order.' The renderer does the rest. This is what Marcus called a clean separation of concerns, and I agree.

Now, Sarah, on intent classification: you are right that it is hard, and I have thought about this. My proposal is that we do not classify intent as a separate LLM step. Instead, the LLM's choice of widgets implicitly determines the shell. If the LLM outputs a single high-density widget like a trace waterfall, the renderer selects the single-focus shell. If it outputs a mix of widgets with a clear primary, it gets the investigation shell. We can define simple heuristics: one widget means single-focus, two to five widgets with a dominant primary means investigation, explicit before/after pairs means comparison. This removes the intent classification problem entirely.

On your 'show all eight widgets' argument: I have data on this. In our screen-share sessions, when an artifact had more than five distinct visualizations, the average time-to-action increased by 140% compared to artifacts with three to four visualizations. SREs told us, in their words, that the artifact was overwhelming and they did not know where to start. Progressive disclosure is not hiding the smoking gun; it is preventing eight non-smoking-guns from burying the one that matters. The LLM's relevance ranking puts the most likely root cause front and center. The detail drawer keeps everything accessible for the SRE who wants to verify.

Round 3: Convergence

Where each persona has shifted and where they still disagree.

Sarah Chen (Product Manager)

I have moved on several points. Priya's screen-share data on cognitive overload is compelling. I was wrong to frame 'show everything' as the safe default. If progressive disclosure genuinely improves time-to-action, that is a user-value argument I cannot ignore. I also accept Marcus's point about validation: a schema that catches errors is better than a stringly-typed template name that fails silently. The 94% first-attempt accuracy number is good enough for me.

I have also come around on the hybrid approach, but with a condition: we build it in two phases. Phase one: convert the existing six templates into two shells, investigation and single-focus, with their current widget sets. This validates the architecture without adding new functionality. Phase two: add the widget catalog and dynamic ranking. This way we ship the architecture incrementally and can bail if phase one reveals problems.

Where I still push back: I do not want four shells in the initial build. Two shells cover 90% of our current usage. Comparison and dashboard are future work. And I want hard metrics on each phase: manifest accuracy rate, fallback rate, and user time-to-action versus the current templates. If hybrid does not beat templates on these metrics, we revert.

My remaining concern is token cost. Marcus, you have not addressed this. A structured manifest is bigger than a template name string. How many additional tokens does the manifest add to each response?

Marcus Rivera (Principal Engineer)

Sarah's phased approach is smart and I endorse it. Converting existing templates into shells first gives us a low-risk validation. I was over-indexing on building the full system at once. On token cost: a typical manifest is 200 to 400 tokens. At current pricing, that is roughly $0.002 to $0.004 per artifact. For context, the entire reasoning chain that produces the investigation is typically 2000 to 5000 tokens. The manifest is noise in comparison.

Priya's proposal to derive shell selection from widget composition is elegant and solves a real problem. I was originally proposing explicit shell selection by the LLM, which is an unnecessary decision point. Letting the renderer infer the shell from the widget set reduces the LLM's decision surface and makes the system more robust. I am adopting this.

I have also changed my mind on density budgets. I initially saw them as a UX concern, but Priya is right that they should be mechanically enforced in the renderer. This makes the rendering pipeline fully deterministic: given a manifest, the output is always the same. No ambiguity, no emergent layout bugs, no 'it looked different on my screen' issues.

Where I still disagree: I think we need three shells in phase one, not two. The single-focus and investigation shells handle different information hierarchies, yes. But the dashboard shell, even a simple version, is needed for the service-health-card use case, which currently accounts for about 20% of artifact renders. Dropping it to phase two means regressing existing functionality.

Priya Nair (UX Designer)

This has been a productive debate. I am glad Sarah pushed on the intent classification problem, because the widget-driven shell selection approach is genuinely better than what I originally proposed. It eliminates an entire class of errors and makes the system simpler to reason about.

I accept Sarah's phased approach with one amendment: I agree with Marcus that we need three shells in phase one, not two. The dashboard shell is structurally simple: it is a grid of equal-weight stat cards and sparklines. Implementing it is probably a day of work, and omitting it would regress the service-health-card experience, which is one of our most-used artifacts.

Where I have softened: the strict five-widget maximum. Sarah was right that a hard cap is a design principle, not a product requirement. The density budget approach is more nuanced and achieves the same goal. If eight widgets all have density score one, their total density is eight, which probably exceeds the budget, but the renderer handles the overflow gracefully via progressive disclosure rather than a hard rejection. The LLM can still rank all eight; the renderer decides how many to show immediately.

My remaining concern is about the widget catalog itself. We need to define a minimum viable set of widgets before we can build anything. I propose seven core widgets for phase one: stat card, sparkline, timeline, table, trace waterfall, log viewer, and change event list. Each needs a defined data contract, density score, and minimum and maximum size constraints. Without this, the manifest schema is theoretical.

Conclusion: Orchestrator Synthesis

Orchestrator Synthesis

1. Decision

The team converges on the hybrid template grammar approach (Option 3), implemented in two phases. The hybrid model separates semantic decisions (what to show, handled by the LLM) from spatial decisions (how to lay it out, handled by a deterministic renderer). The LLM outputs a JSON manifest specifying an ordered list of typed widgets with data payloads. The renderer infers the appropriate shell from widget composition, places widgets into named slots by priority, and enforces information density budgets through progressive disclosure.

This approach was chosen over pure templates (which cannot handle the combinatorial space of observability queries and have no graceful degradation for novel query types) and pure widget ranking (which requires the LLM to make spatial layout decisions it is unreliable at). The hybrid gives us deterministic rendering, schema validation, clear system boundaries, and a path to dynamic content without sacrificing visual consistency.

2. Architecture Specification

Shells (Phase 1: three shells)

> Single-Focus: header (1 widget) + primary (1 widget) + detail (0-N, collapsed). Triggered when the manifest contains a single high-density widget. Use case: trace waterfall deep-dive, single metric exploration.
>
> Investigation: header (1) + primary (1-2) + context (0-3) + detail (0-N, collapsed). Triggered for mixed widget sets with a dominant primary. Use case: root cause analysis, latency investigation.
>
> Dashboard: header (1) + grid (2-6 equal-weight widgets). Triggered for homogeneous widget sets with no dominant primary. Use case: service health overview, SLO status.

Widget Catalog (Phase 1: seven widgets)

Widget	Density	Min Size	Slot Affinity
Stat Card	1	1x1	header, grid
Sparkline	1	2x1	context, grid
Timeline	2	3x1	context, primary
Table	2	3x2	primary, context
Trace Waterfall	3	4x2	primary
Log Viewer	2	3x2	primary, detail
Change Event List	1	2x1	context, detail

JSON Manifest Schema (simplified)

```
{
```

```
"version": "1.0",
"metadata": {
"title": "string",
"severity": "critical | warning | info",
"query_intent": "string (for logging)"
},
"widgets": [
{
"type": "stat_card | sparkline | timeline | ...",
"priority": "number (1 = highest)",
"data": { "...widget-type-specific payload" },
"display_hints": {
"size_preference": "compact | default | expanded",
"emphasis": "primary | secondary | tertiary"
}
}
]
}
```

Note: the shell is not specified in the manifest. The renderer infers it from widget composition using deterministic heuristics (single high-density widget implies single-focus; mixed set with emphasis=primary implies investigation; homogeneous density-1 set implies dashboard).

Renderer Contract

The renderer is a pure function: given a valid manifest, it always produces the same HTML output. It validates the manifest against the schema, infers the shell, places widgets into slots by priority order, enforces the density budget (default: 8 points for investigation, 6 for single-focus, 10 for dashboard), and collapses overflow widgets into the detail drawer. Invalid manifests trigger a fallback to the investigation shell with a single table widget showing raw data.

3. Implementation Plan

Phase 1: Architecture Validation (2 sprints)

> Define the manifest JSON schema and write validators. Build the three shell layouts (single-focus, investigation, dashboard) with named slots and density enforcement. Port the existing six templates into manifests that target these shells. Validate that existing artifacts render identically. Measure: manifest accuracy rate, fallback rate, rendering parity with current templates.

Phase 2: Dynamic Widget Catalog (2 sprints)

> Build the seven core widgets as independent, tested components. Update the LLM prompt to output dynamic manifests instead of template selections. Implement the shell-inference heuristics. Implement progressive disclosure and the detail drawer. A/B test hybrid vs. current templates on time-to-action and user satisfaction.

Phase 3: Expansion (future)

Add the comparison shell. Expand the widget catalog (dependency graph, deployment timeline, cost widget). Allow user customization of density budgets and default shell preferences.

4. Open Questions

1. Widget data contracts: what is the exact schema for each widget type's data payload? This requires alignment between the backend data pipeline and the frontend components.

2. Shell-inference edge cases: what happens when the widget set is ambiguous? For example, two density-2 widgets with no clear primary. Does this default to investigation or dashboard?

3. Responsive behavior: how do shells adapt to narrow viewports (mobile, side panel)? Does the density budget change at smaller breakpoints?

4. Caching and persistence: should manifests be cached so users can revisit a previous artifact? If so, what is the cache key and TTL?

5. LLM prompt engineering: how much of the widget catalog and shell documentation needs to be in the system prompt versus retrievable via tool use?

6. Accessibility: do all widgets meet WCAG 2.1 AA? Who owns accessibility testing for each widget?

5. Risk Register

Risk	Likelihood	Impact	Mitigation
LLM produces invalid manifests at scale	Medium	High	Schema validation + retry loop + investigation-shell fallback. Monitor fallback
Shell-inference heuristics produce wrong layout	Medium	Medium	Log inferred vs. expected shell in telemetry. Allow manual override in the ma
Widget catalog grows faster than test coverage	High	Medium	Each widget must ship with a Storybook story, snapshot tests, and accessibi
Phase 1 takes longer than 2 sprints	Medium	High	Scope-cut dashboard shell to phase 2 if needed. Single-focus + investigation
Progressive disclosure hides critical signals	Low	High	A/B test with SREs. Track whether collapsed widgets are expanded in incide
Token cost of manifests impacts latency	Low	Low	Manifests are 200-400 tokens, negligible versus the 2000-5000 token reason